



IndusAlexa And Google Assistant-Enabled IoT CONTROLLER

ASHWINI KUMAR SINHA

AI smart speakers and AI-enabled home automation devices are widely adopted and continue to see strong demand. However, achieving seamless integration with a high level of customisation remains challenging, particularly when adding sensors, lights, switches, relay actuators, fans, and bulbs to meet specific requirements. In practice, smart bulbs, switches, and sensors are typically sold as

standalone devices. No single customisable platform enables sensors, bulbs, fans, actuators, and displays to operate together as a unified, centralised system. As a result, multiple independent devices must be connected and integrated, making automation management complex. This challenge is further intensified by the reliance on multiple mobile applications and device-specific notifications, with no consolidated

control interface available.

Designing a custom IoT controller introduces additional complexity, particularly when integration with AI smart speakers such as Alexa, Apple Home, or Google Assistant is required. Native integration options are limited and often rely on third-party, API-based solutions. Seamless provisioning, device reset, and over-the-air firmware management are also difficult to implement reliably and frequently remain at a prototype stage rather than being suitable for production deployment.

This system addresses these challenges by using the IndusBoard Coin and leveraging its compatibility with the RainMaker platform. RainMaker enables a production-ready system design with wireless firmware management, device monitoring, and provisioning. It allows Wi-Fi credentials to be configured wirelessly, in a manner aligned with commercially available smart home devices.

The approach delivers a complete manufacturer-to-client endpoint solution while enabling seamless integration with Apple Home, Google Assistant, and Alexa for smart assistant-based automation. The IndusBoard Coin provides

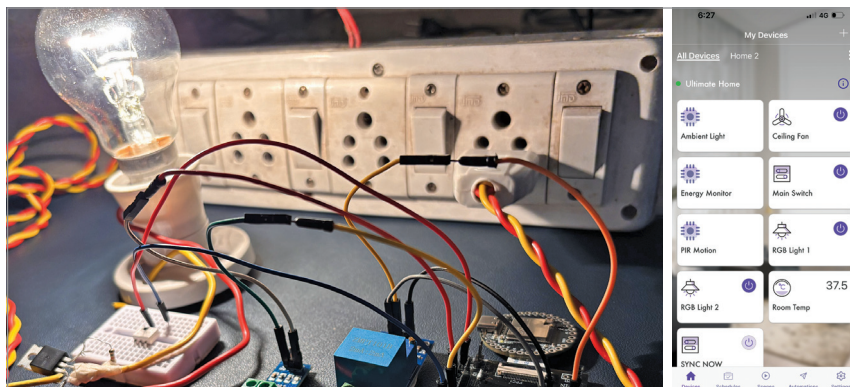


Fig. 1: Author's prototype

```
1 // ESP RainMaker + FastLED - Full RGB Lights + Fan + Switch + Touch + GPIO0 Reset (EXACTLY like original)
2 #include "RMaker.h"
3 #include "WiFi.h"
4 #include "WiFiProv.h"
5 #include "AppInsights.h"
6 #include <FastLED.h>
7
8 #define TOUCH_THRESHOLD 35
9
10 const char *service_name = "PROV_RGB_HOME";
11 const char *pop = "abcd1234";
12
13 // MUST be constexpr or #define for FastLED template
14 constexpr uint8_t LIGHT1_PIN = 11; // WS2812B Data - Light 1
15 constexpr uint8_t LIGHT2_PIN = 33; // WS2812B Data - Light 2
16
17 static int gpio_reset = 0; // GPIO0 - Boot button (handled exactly like official example)
18 static int fan_pin = 10;
19 static int switch_pin = 41;
20
21 // Touch pins
22 static int touch_fan = 1; // T1
23 static int touch_light1 = 2; // T2
24 static int touch_light2 = 4; // T4
25
26 // FastLED arrays
27 #define NUM_LEDS 1
28 CRGB leds1[NUM_LEDS];
29 CRGB leds2[NUM_LEDS];
30
```

Fig. 2: Code snippet showing the configuration of the RainMaker device

BILL OF MATERIALS

Name	Designator	Quantity
IndusBoard Coin	MOD1	1
Relay module	RL1-RL3	3
Fan speed controller module (F1)	F1	1
Neo pixel LED light	L1, L2	2
3.3V AC-DC power module	MOD2	1